

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of FRIDAY 17 JANUARY 2025 – Laboratory Part

WITH SOLUTIONS

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory syntax and semantics of languages, divided in four sections: (75% of the final grade)

- 1) regular expressions and finite automata (1 exercise × 10 points)
- 2) syntax analysis and parsing methodologies (1 exercise × 10 points)
- 3) grammar design – translation – semantic analysis (2 exercises × 5 points)
- 4) *bonus* question (1 exercise × 5 points)

Lab compiler design by Flex and Bison (25% of the final grade)

- 1) basic questions (30 points)
- 2) *bonus* question (5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Basic Questions (30 points)

1. Consider the implementation of the Acse compiler shown in class.

Modify the specification of the lexical analyser (flex input) and the syntactic analyser (bison input) and any other source file required to extend the Lance language with an implementation of the **ternary operator**.

The ternary operator has the following syntax:

$$\langle exp.1 \rangle ? \langle exp.2 \rangle : \langle exp.3 \rangle$$

It takes three parameters: a **condition** ($\langle exp.1 \rangle$), a **true value** ($\langle exp.2 \rangle$) and a **false value** ($\langle exp.3 \rangle$). The result of the operator is equal to the true value if the condition is true (i.e. if the value of the condition is non-zero), otherwise it is equal to the false value. The priority of the ternary operator is the lowest of all operators, and it is left-associative.

The following code snippet exemplifies the use of the operator.

```
int a;
a = 10;
a = (a > 5 ? 5 : 15) + 3;
write(a == 8 ? 1 : 0); // prints 1
```

- (a) Define the tokens (and the related declarations in **scanner.l** and **parser.y**). (3 points)
- (b) Define the syntactic rules or the modifications required to the existing ones. (4 points)
- (c) Define the semantic actions needed to implement the required functionality. (18 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/parser.y b/acse/parser.y
--- a/acse/parser.y
+++ b/acse/parser.y
@@ -70,6 +70,7 @@ void yyerror(const char *msg)
 %token TYPE
 %token RETURN
 %token READ WRITE ELSE
+%token QUESTION COLON

// These are the tokens with a semantic value.
%token <ifStmt> IF
@@ -96,6 +97,7 @@ void yyerror(const char *msg)
 * specific keyword used (%left, %right).
 */

+%left QUESTION COLON
%left OROR
%left ANDAND
%left OR_OP
@@ -435,6 +437,28 @@ exp
 $$ = getNewRegister(program);
 genOR(program, $$, rNormalizedOp1, rNormalizedOp2);
 }
+ | exp QUESTION exp COLON exp
```

```

+ {
+     // Reserve a new register for the result of the operation
+     $$ = getNewRegister(program);
+     // Generate a branch to the 'false' case if the condition is equal to zero
+     t_label *lElse = createLabel(program);
+     genBEQ(program, $1, REG_0, lElse);
+     // Generate the assignment of the true value to the result.
+     // The instruction generated here is reached if the previous branch was
+     // not taken.
+     genADD(program, $$, $3, REG_0);
+     // Generate an unconditional jump that skips over the code for the 'false'
+     // case.
+     t_label *lExit = createLabel(program);
+     genJ(program, lExit);
+     // Generate the label for the 'false' case
+     assignLabel(program, lElse);
+     // Generate the assignment of the false value to the result.
+     genADD(program, $$, $5, REG_0);
+     // Generate the label necessary for skipping the 'false' case.
+     assignLabel(program, lExit);
+ }
+
;

```

```

var_id
diff --git a/acse/scanner.l b/acse/scanner.l
--- a/acse/scanner.l
+++ b/acse/scanner.l
@@ -72,6 +72,8 @@ ID [a-zA-Z_][a-zA-Z0-9_]*
" && " { return ANDAND; }
" || " { return OROR; }
" , " { return COMMA; }
+ "?" { return QUESTION; }
+ ":" { return COLON; }

"do" { return DO; }
"else" { return ELSE; }
diff --git a/tests/ternary/ternary.src b/tests/ternary/ternary.src
new file mode 100644
--- /dev/null
+++ b/tests/ternary/ternary.src
@@ -0,0 +1,6 @@
+int a;
+a = 10;
+a = (a > 5 ? 5 : 15) + 3;
+write(a == 8 ? 1 : 0); // prints 1
+read(a);
+write(a > 0 ? 1 : 0);

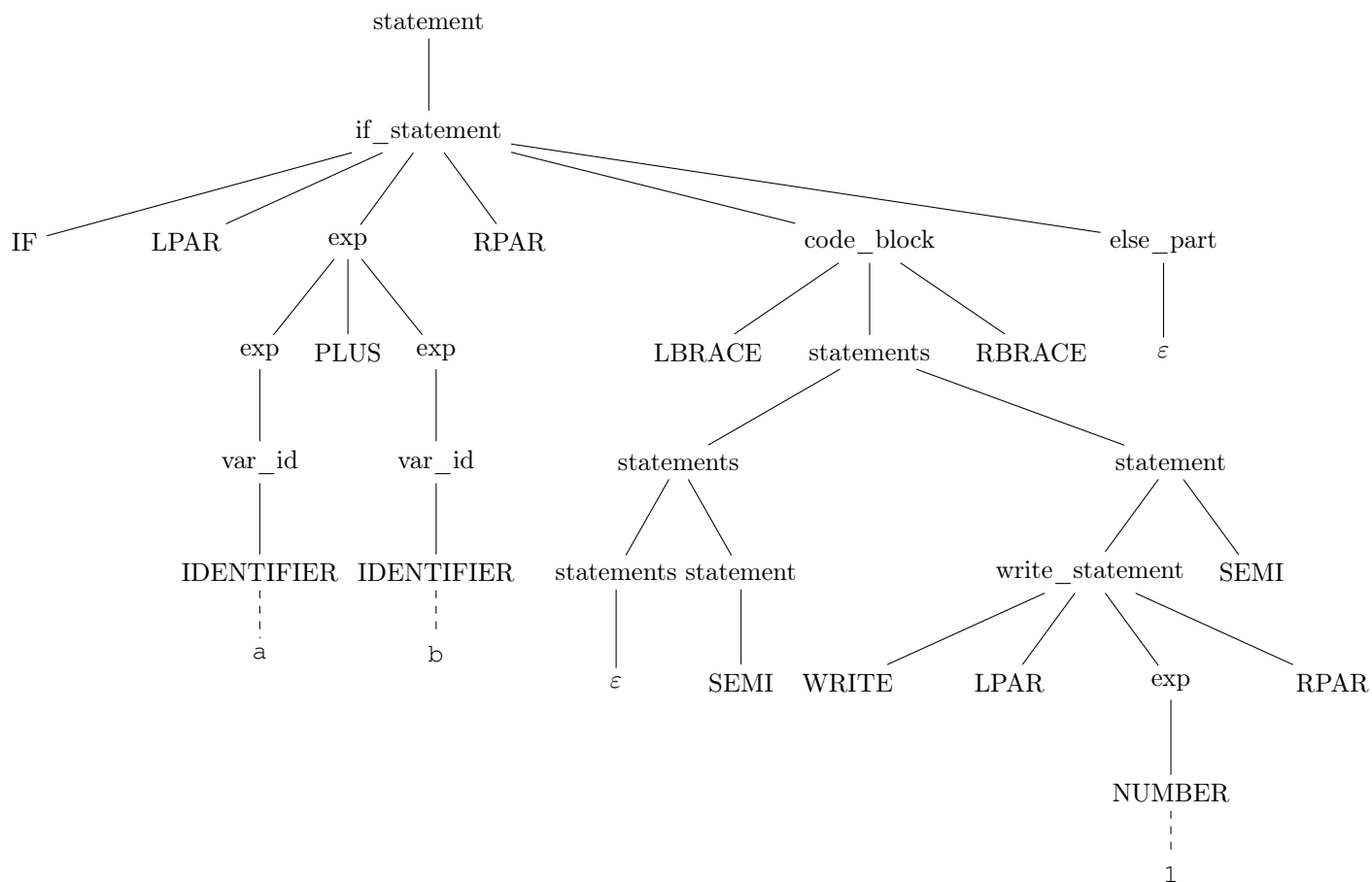
```

2. Given the following Lance code snippet:

```
if (a+b) {write (1); }
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the statement nonterminal*. (5 points)

Solution



2 Bonus Question (5 points)

1. Explain how to modify the answer given to the questions about modifications to ACSE in order to support a shortened version of the ternary operator where the true value is not specified and is implicitly equal to the condition.

Show both the grammar rules necessary for this modification, and describe in plain English words the logic of the semantic actions required.
